



Examining and detecting academic misconduct in written documents using revision save identifier numbers in MS Word as exemplified by multiple scenarios

Dirk HR. Spennemann^{a,*}, Rudolf J. Spennemann^b, Clare L. Singh^{c,d}

^a School of Agricultural, Environmental and Veterinary Sciences, Charles Sturt University, PO Box 789, Albury, NSW, 2640, Australia

^b Independent Researcher, Albury, NSW, 2640, Australia

^c School of Dentistry and Medical Sciences, Charles Sturt University, Locked Bag 588, Wagga Wagga, NSW, 2678, Australia

^d Academic Misconduct Team, Faculty of Science and Health, Charles Sturt University, Australia

ARTICLE INFO

Keywords:

Academic misconduct
AI-Generated text
Authorship
Contract cheating
Document creation and editing
Digital forensics
Plagiarism

ABSTRACT

Deliberate academic misconduct by students often relies on the use of segments of externally authored text, generated either by commercial contract authoring services or by generative Artificial intelligence language models. While revision save identifier (rsid) numbers in Microsoft Word files are associated with edit and save actions of a document, MS Word does not adhere to the ECMA specifications for the Office Open XML. Existing literature shows that digital forensics using rsid requires access to multiple document versions or the user's machine. In cases of academic misconduct allegations usually only the submitted files are available for digital forensic examination, coupled with assertions by the alleged perpetrators about the document generation and editing process. This paper represents a detailed exploratory study that provides educators and digital forensic scientists with tools to examine a single document for the veracity of various commonly asserted scenarios of document generation and editing. It is based on a series of experiments that ascertained whether and how common edit and document generation actions such as copy, paste, insertion of blocks of texts from other documents, leave tell-tale traces in the rsid encoding that is embedded in all MS Word documents. While digital forensics can illuminate document generation processes, the actions that led to these may have innocuous explanations. In consequence, this paper also provides academic misconduct investigators with a set of prompts to guide the interview with alleged perpetrators to glean the information required for cross-correlation with observations based on the rsid data.

1. Introduction

Academic misconduct by students has increasingly become a problem, both in terms of outright plagiarism using sources readily available on the internet (Park, 2003; Awasthi, 2019) and in terms of commercially written material ('contract cheating'), commonly disguised as 'sample essays' (Newton, 2018; Ahsan et al., 2022; Bretag et al., 2019a; Hill et al., 2021). The recent rise of publicly accessible generative Artificial intelligence (genAI) tools such as ChatGPT allow students to readily and effortlessly generate text for their assignments (Perkins et al., 2023; Weber-Wulff et al., 2023; Spennemann, 2023a; Cotton et al., 2023; Naidu and Sevnanarayan, 2023). Conditions for abuse and academic misconduct have never been 'better.' This is not the venue to discuss the motivations and educational/societal conditions for such misconduct

(Miles et al., 2022). As such misconduct threatens to undermine the integrity of the degrees and qualifications attained, all universities have processes in place to address academic misconduct through education and awareness training; through deterrence via detection and (graded) punitive action (Campbell and Waddington, 2020; Miron et al., 2021); and through modifications to assessment design (Bretag et al., 2019b; Harper et al., 2021; Chaudhry et al., 2023).

Current plagiarism detection software (e.g. Turnitin) compare a submitted text with online sources as well as other documents submitted to the service. This method can detect verbatim plagiarism, but is less effective in detecting plagiarised text that has been 'washed' through online paraphrasing tools (e.g. <https://www.wordtune.com>, <https://quillbot.com>). Texts written by genAI follow the same style of writing and expression, but will differ sufficiently in content to avoid software-based

* Corresponding author.

E-mail addresses: dsPennemann@csu.edu.au (D.H.R. Spennemann), rudolf@spennemann.net (R.J. Spennemann), csingh@csu.edu.au (C.L. Singh).

<https://doi.org/10.1016/j.fsidi.2024.301821>

Received 9 October 2023; Received in revised form 8 July 2024; Accepted 31 August 2024

Available online 10 September 2024

2666-2817/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

plagiarism detection (Spennemann, 2023b). Likewise, essay/assignment text that was written and supplied by other authors (commercial or not) escapes automated detection. Consequently, tools are needed that can be used by academics to i) identify submissions as suspicious and ii) interrogate these with regard to document authorship and genesis.

Building on crime prevention theory, Clare et al. noted that contract cheating and academic misconduct display patterns of non-randomness across offenders (i.e. submitters of work) and targets (i.e. institutions and specific teaching staff), which allowed them to develop suggested rule sets that can aid in the identification of potentially suspicious submissions (Clare et al., 2017). Among other indicators they include flags such as discrepant marks across assignments within and between subjects/courses taken by the student; variations in language and expression as well as changes in paragraph structure between other examples of student's writing; and inconsistencies in formatting and type fonts within the same document (Spennemann et al., 2024).

Once flagged as suspicious, such work then needs to be subjected to a preliminary digital forensics investigation which examines the document for traces that are indicative of or may indicate the work as non-genuine. As this cannot be done from PDF versions, the original MS Word file will be required. A preliminary digital forensics investigation involves an examination of the document properties such as document authorship, creation date and time, total editing time, and number of edits. If inconsistencies are found, or if the initial inconsistencies (e.g.

font etc) are deemed persuasive, a more detailed analysis is required.

This process needs to query the person who submitted the work with regard to the writing and editing process they followed with the document in question, which can then be cross-checked with a detailed digital forensics investigation (Fig. 1). The aim of any digital forensics investigation is to find what Jeong & Lee coined the 5WH1 (what, who, when, where, why and how) (Jeong and Lee, 2017). It is at that stage in the process that a detailed understanding of the nature and specific properties embedded in MS Word documents comes into play. MS Word files retain a considerable amount of information that can illuminate the document generation process. While it may not be able to answer the where and why of the modification, document forensics can illuminate the what and how, with some, albeit more limited, additional information on the who and when. This should be augmented by a structured interview with the alleged perpetrator of academic misconduct which elicits a detail set of information about the processes the document author followed in the generation of the document in question. Across-correlation of these assertions with the observations based on the rsid data will show congruence (probably innocuous) or dissonance (misconduct).

This paper will provide a discussion of several of these specific properties with special emphasis on their usefulness in detecting and identifying editing processes over the life-history of a specific document. While we are aware that specialised users may use the Linux-based

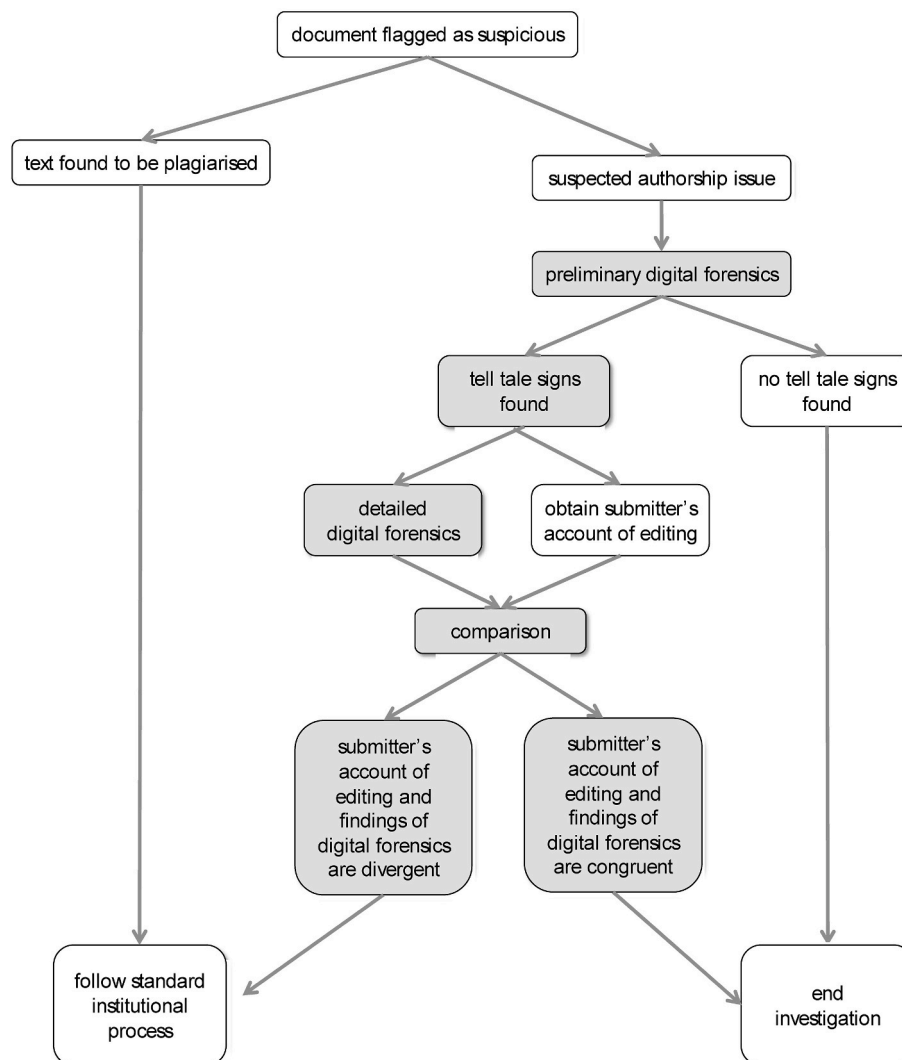


Fig. 1. Process flow of a standard academic misconduct investigation. This paper will deal with the process steps shown as grey boxes.

LibreOffice, or Apache’s OpenOffice software, to generate their files, we are focussing this current analysis on MS Word, as this is bar far the most ubiquitous word processing software used by students and office workers. We will first provide a discussion of the nature and key features of revision save identifiers (Rsid) embedded in the format of MS Word documents, and then describe a number of document creation and file management experiments that emulate common author practices to test the usefulness of rsid numbers in correctly identifying these practices. Building on the results of these experiments, the authors will then discuss the role rsid analysis can play in investigations conducted by academic integrity officers in common scenarios where the editorial actions asserted by authors have come into question. The paper concludes with a discussion of suitability of rsid for examining and detecting document manipulations in general and in the context of academic misconduct investigations in general.

1.1. The file structure of MS word files

The Microsoft Office suite of applications has dominated the word-processing and document editing market on personal computers and tablets since the early 1990s (Mahlow and Piotrowski, 2022). The Office Open XML standard (ECMA, 2016; ISO/IEC, 2008) has been implemented in MS Word since 2007. Under this standard, all document components (text, styles, settings, meta data) are stored in a compressed archive, visible to the general user as a.docx file. Each new document is allocated a revision save identifier (rsid) at the point of document creation (*rootrsid*). This *rootrsid* persists when the file is copied, either through a ‘save as’ action in the editing software, or through copying the file at the operating system level (MacOs, Windows). Every document editing session that ends with a save action (or an e-mail send) is assigned an additional unique rsid number. These numbers, expressed as eight hexadecimal digits, are collectively stored in a summary table (*‘rsidtbl’*) in the file settings.XML (in the word subfolder) as a sequence in ascending numerical order (Spennemann and Singh, 2024). In the file document.XML (also in the word subfolder), each session rsid is then associated (in the form of enclosing tags, Table 1) with specific actions such as text insertions, deletions (if track changes was activated), and specific formatting such as the insertion of page and section breaks as well as subsections of tables.

The relevant standards (ECMA and ISO/IEC 29500–1:2016) stipulate that rsid should be randomly generated but with increasing numerical value, thereby documenting the progress of the editing (ECMA, 2016; ISO/IEC, 2008; ISO/IEC, 2016). A prior paper by the authors discussed the generation of rsid numbers and could show that while MS Word allocates unique and persistent rsid for each newly created document (*rootrsid*) as well as one persistent rsid for each edit session that is completed with a save action, it does not correctly implement these standards. The processes of generation and transmission of rsid numbers between different editing states within MS Word files have been discussed elsewhere and need not be repeated here in detail (Spennemann and Singh, 2024). Suffice to say that any newly allocated rsid may be smaller or larger than the document rsid (*rootrsid*) and the rsid of the preceding edit session (Spennemann and Singh, 2024).

Table 1
Types of tags with rsid numbers discussed in this paper.

Type	Meaning
charrsid	rsid allocated to text characters (for example to reflect a change in type font)
insrsid	rsid allocated at the point where text has been inserted
pararsid	rsid allocated to a newly created paragraph
rootrsid	rsid allocated at the point of creation of the document itself
rsidRPr	rsid allocated to the formatting of section mark
rsidR	rsid allocated to the formatting of the text of the text section mark
rsidRDefault	rsid allocated as the default rsid for runs of a paragraph (copied from other files)

1.1.1. Probabilities

MS Word allocates rsid numbers that can be up to eight digits long which equates to 10^8-1 possible combinations (assuming that the overall number 0 is not a viable option). As a newly created Word document contains one document rsid (*rootrsid*) as well as one persistent rsid that is allocated to the first save action, $10^{15}-2 \times 10^7$ possible rsid combinations exist. Assuming the file has seen one further editing phase ending in a save, which adds a further persistent rsid, the number of possible rsid combinations equates to $10^{23}-3 \times 10^{15}$ (approx.). Thus the probability that two newly created Word documents share same the rsid combination by chance is close to one in a quadrillion, with the probability reducing to one in one hundred sextillion with one further editing phase ending in a save. In short, the probability of two documents having the same rsid combinations is infinitesimally small.

While theoretically possible, the probability that two independently generated files have the same rsid number allocated to an edit&save session is close to one in one hundred million (10^8-1)—effectively 28 times smaller than the probability of winning first division in the lottery. This could theoretically occur, for example, where text is copied from one file to another with the rsid (in the *pararsid* or *charrsid* tags) retained in the copy process. The moment the chunk of copied text originates from a source text with more than edit & save actions, the number of rsid potentially carried over increases, while the probability of rsid combinations being the same by chance decreases exponentially. Thus, in theory, a document comparison between two independently submitted pieces of work should show whether and where copying occurred.

1.1.2. rsid numbers and version tracking in the literature

The usefulness and interpretation of rsid information for digital document forensics has been discussed mainly on a purely theoretical basis several authors (Didriksen, 2014; Ries, 2017; Garfinkel and Migletz, 2009; Fu et al., 2011, 2015; Fu and Sun, 2015; Joun et al., 2018, 2021; Langweg, 2012; Castiglione et al., 2017). Numerous approaches require access to the machine on which the file was generated or is stored (Jeong and Lee, 2017; Choi et al., 2021) while others require other document versions for comparative analysis (Spennemann and Spennemann, 2023). In the arena of academic plagiarism, however, it is usually only a single file that is available for analysis. Joun et al. looked at the existence of rsid in various word processing programs but did not examine the rsid allocation and persistence characteristics in any greater detail (Joun et al., 2021). Johnson & Davies, for example, postulated that rsid values might be useful in detecting or identifying contract cheating but did not elaborate of the mechanics of the analysis (Johnson and Davies, 2020) The authors have noted the potential usefulness of rsid numbers for the detection of academic plagiarism, in particular contract cheating (Johnson and Davies, 2020; Johnson et al., 2020, 2022). The authors commented that rsid numbers might be a tool to determine the chronological sequence of editing a given text. Theirs was a plausible assumption, given that the ECMA specifications for the Office Open XML standard stipulate that “every editing session shall be assigned a revision save ID that is larger than all earlier ones in the same file” (ECMA, 2016). The authors expanded on this by developing an (unpublished) tool that flagged whether a document text had been pasted as text chunk or whether successive editing had occurred in a document. While Johnson and Davies could show that rsid tags indeed signal the insertion of text between save events, and thus can be deployed to detect changes, but they could not demonstrate the validity of the chronology (Johnson and Davies, 2020; Johnson et al., 2020).

Fundamental to all papers is the assumption that Microsoft adhered to the implementation of the relevant standards (ECMA and ISO/IEC 29500–1:2016). As shown by the present authors, this is not the case (Spennemann and Singh, 2024). In consequence it is necessary to assess experimentally which of the editing processes leave tell-tale traces and what these traces look like. Some experimental work has been carried out (Spennemann and Singh, 2024; Spennemann and Spennemann, 2023). Experimental work by the authors confirmed that MS Word

allocates a unique and persistent *rsid* (*rootrsid*) for each newly created document and that these, as well as the *rsid* allocated to edit&save actions (as stored in the *rsidtbl* listing), will persist even when files are copied at the operating systems level or when copied via a 'save as' action (Spennemann and Singh, 2024). Given that these *rootrsid* will be retained even if these copied files diverge in subsequent edit&save sessions, an given that the legacy *rsid* the *rootrsid* will also be retained, it is possible to develop document genealogies, showing common ancestor documents and branching from a common stem. This could be demonstrated experimentally by analysis over 400 MS Word templates for journal titles issued by an academic publishing house (Spennemann and Spennemann, 2023).

This paper extends the experimental work by emulating common author practices in the generation of documents such as copy, paste, moving sections of text, and insertion of blocks of texts from other documents.

2. Materials and methods

A number of experiments were designed to simulate and examine the effects of a range of common file management and document editing actions have on the allocation of *rsid* numbers in MS Word files.

2.1. Accessing *rsid* numbers

In the file settings.xml, the tag '*rsidroot*' identifies the *rsid* allocated to the document itself. A summary table ('*rsidtbl*') stores *all* *rsid* numbers contained in a given document and lists them in increasing numerical order (Spennemann and Singh, 2024).

There are two fundamental ways of accessing and extracting the *rsid* information contained in MS Word files: by opening (unpacking) the .docx compressed archive file and examining the XML files which store the *rsid* numbers in hexadecimal format, or by examining the document as exported in rich text format (rtf) (Spennemann and Singh, 2024). This paper employs the second method (for process see Appendix A).

2.2. Revision experiments

2.2.1. Revision experiment 1

Revision Experiment 1 assesses whether pre-existing *rsid* influence the number of revisions as displayed in the property window of MS Word files (File > Properties > Statistics). To this effect a new file was created on the author's (DHRS) MacBook (MS Word 16.54 build 21101001), which has in excess of 4000 *rsid* in the .dot/.dotm template file (Spennemann and Singh, 2024). Text was added to the newly created file followed by a save. In total ten iterations of text adding and saving were carried out. At each step, the 'statistics' panel of the 'properties' window was queried to ascertain the number of recorded revisions.

2.2.2. Revision experiment 2

Revision Experiment 2 assesses whether save actions without edits influence the recorded number of revisions as displayed in the property window of MS Word files. As base file served the document created for the Revision Experiment 1, which was saved under new name. Multiple save actions were performed, without changing any of the text. At each step, the 'statistics' panel of the 'properties' window was queried to ascertain the number of recorded revisions.

2.2.3. Revision experiment 3

Revision Experiment 3 examines whether times of inaction would influence the total document editing time as displayed in the property window of MS Word files. As base file served the document created for the Revision Experiment 1, which was saved under new name. That file was left open, with the 'statistics' panel of the 'properties' window queried after 80 min had passed (experiment 3A). A duplicate base file was opened and some text was added immediately, after which the file

was left open the without a save action. The 'statistics' panel of the 'properties' window was queried after 20 min had passed.

2.2.4. Revision experiment 4

Revision Experiment 4 examines whether the total document editing time as displayed in the property window of MS Word files reflects actual editing time. As base file served a blank document created on the author's (DHRS) MacBook (MS Word 16.54 build 21101001). Data were saved at 1-min intervals for five iterations, 2-min intervals for three iterations, 5-min intervals for two iterations followed by one 10-min interval. At each step, the 'statistics' panel of the 'properties' window was queried to ascertain the number of recorded revisions and the elapsed document editing time.

2.2.5. Revision experiment 5

Based on the unexplained increases in the number of revisions (as displayed in the property window of MS Word files) during revision experiment 4, a new file was created and subjected to series document processing events, each of which was followed by a period of time of inaction in the that file, while edit and save actions occurred in another document that was being worked on at the same time (in a different window). The events tested were 'edit—wait period—save;', 'edit—wait period—edit—save;', 'edit—e-mail send—save;', and 'edit—wait period—e-mail send—save.' The 'statistics' panel of the 'properties' window was queried at each action (experiment 5A). To test whether the number of revisions would carry over to a file created via a 'save as' action, upon conclusion of experiment 5A the file was saved under a different name and additional edit&save actions performed (experiment 5B).

2.2.6. Revision experiment 6

The experiment was designed to assess whether the use of a legacy file would carry over the number of revisions and total document editing time data, or whether the total editing time would reflect the actual total time elapsed between the last document save and the current edit. A copy of a document that had been last saved on 28 January 2021 with two revisions and 3 min editing time was opened on 11 October 2022, then blanked and saved (6A). This was replicated with a copy of a document last saved on 11 October 2022, with 156 revisions and 196 min editing time (6B). At each step, the 'statistics' panel of the 'properties' window was queried to ascertain the number of recorded revisions and the elapsed document editing time.

2.3. *rsid* experiments

In total, ten *rsid* experiments were carried out to assess the effects of different text generation and edit events (see Table 4 for overview).

2.3.1. *rsid* experiment 1

RsId Experiment 1 examines differences between iterative generation of text in a single document and the insertion of iteratively generated text by copying or by read-in of the data. A set of three files was generated on a MacBook with MS Word 2011 (version 14.7.7) (experiment 1A) and a set of three files on a Windows PC using MS Word 2013 (version 15.0) (experiment 1B).

As base document served a file developed for and reported in a previous paper (blank MS Word file created on a MacBook, using MS Word 2011 [version 14.7.], saved, opened and text pasted in followed by a save & close action) (Spennemann and Singh, 2024). File 1 of each set comprised of the base document, which was opened and saved to rtf (to serve as control) file 2 comprised of the base document, which was opened, all content deleted (without a save action) and in the entire iteratively generated control text (see below for explanation) was pasted into the file via the clip board. This was followed by save action and save to rtf action. The process for file 3 was the same as that for file 2, with the exception that the iteratively generated control text was not pasted in but read in using the 'insert file' option.

The *iteratively generated control text* was text taken from a file with edit state 10 of experiment 3A as reported in a previous paper (Spennemann and Singh, 2024). The file was generated by opening a copy of base document, pasting in the derived text and saving it under a new file name. That file was reopened, pasting in further text, and saving it under a new file name. This continued iteratively until the ninth iteration. The final process involved adding lines between each of the pasted texts, resulting a file with 10 edit stages (Spennemann and Singh, 2024) with a complex rsid structure (Fig. 2).

2.3.2. rsid experiment 2

Similar to the previous experiment, rsid experiment 2 examines differences between the insertion of prior generated text, but compares files with and without Endnote referencing data (again by copying or by reading-in of the entire file).

As base document served a file for developed and reported in a previous paper (blank MS Word file created on a MacBook, using MS Word 2011 [version 14.7.] followed by a save & close action) (Spennemann and Singh, 2024). The control text was comprised of a chunk of iteratively generated text taken from one of the author's (DHRS) manuscripts then in progress. Two files were generated, one with the embedded Endnote references, and one where these references were deleted before the first save action. One set of files was generated on a MacBook and the other on a Windows machine (using the same, MacOS generated manuscript file). The generated control text in Windows format had 3 rsid listed in its *rsidtbl*, while the file in Mac format had 4772 rsid listed in *rsidtbl*. The high number of rsid is due to legacy data embedded.dot template file on the author's MacBook (see supportive document for explanation).

The test experiment was carried out on a MacBook (experiment 2A, 2B) and on a Windows machine (experiment 2C, 2D). Where the copied (or inserted) files had embedded Endnote references, Endnote was updated before the first save action, updated again and then saved directly to rtf. Table 2 shows the combinations tested.

2.3.3. rsid experiment 3

Rsid Experiment 3 assesses whether it is possible to detect differences between text that was *typed in* as opposed to text that was *pasted in* within the same edit session (before a save). As base document served a file for developed and reported in a previous paper (blank MS Word file

insrsid3551197	Edit session 1		
insrsid2692646			
insrsid10497994		pararsid10497994	
insrsid10497994	Edit session 2		insrsid2779502
insrsid10497994		pararsid16414681	
insrsid16414681	Edit session 3	pararsid2779502	insrsid2779502
insrsid10497994		pararsid5978667	
insrsid5978667	Edit session 4	pararsid2779502	insrsid2779502
insrsid2779502			
insrsid16414681		pararsid870953	
insrsid870953	Edit session 5	pararsid2779502	insrsid2779502
insrsid5978667		pararsid14813932	
insrsid14813932	Edit session 6	pararsid2779502	insrsid2779502
insrsid870953		pararsid2779502	
insrsid138996	Edit session 7		insrsid2779502
insrsid2779502		pararsid138996	
insrsid138996			
insrsid14813932		pararsid16319538	
insrsid16319538	Edit session 8	pararsid2779502	insrsid2779502
insrsid138996		pararsid3828268	
insrsid3828268	Edit session 9	pararsid2779502	insrsid2779502

Fig. 2. rsid structure of the iteratively generated control text (Spennemann and Singh, 2024). The rsid 2779502 is associated with the insertion of lines in edit session 10.

Table 2

Combinations of experiment 2.

Action	File generated on	no Endnote		Endnote	
		MacOS	Windows	MacOS	Windows
Copy text via clipboard	MacOS	2A1	2B1	2A3	2B3
	Windows	2A2	2B2	2A4	2B4
Read in file via 'insert'	MacOS	2C1	2D1	2C3	2D3
	Windows	2C2	2D2	2C4	2D4

created on a MacBook, using MS Word 2011 [version 14.7.] followed by a save & close action) (Spennemann and Singh, 2024). This was opened and saved after two zones (for typed text and pasted text) had been established (separated by a row of '=' signs). That file was opened, with text inserted by typing and by pasting. The pasted text had been copied from other source but had been 'washed' by pasting it first into a text editor thereby stripping all residual xml edit codes. After a save, that process was repeated.

2.3.4. rsid experiment 4

Rsid Experiment 4 examines the effects of swapping files between computers of different operating systems. Two variations were tested. The iterative, but sequential generation of text by adding a new line of text at each stage (experiment 3A) and the iterative generation of text, where at each stage, with track changes turned on, text was deleted, text was added, and the file then saved. The sample files were copied onto a USB stick which was shuttled between a MacBook using MS Word 16.38 and a Windows PC using MS Word 2008 (build 13127.21624). In total there were five iterations per operating system.

2.3.5. rsid experiment 5

Rsid Experiment 5 examines the effects of a complex series of file creation and editing processes (see Table 3 for details) on the allocation and persistence of rsid. As base document served a file for developed and reported in a previous paper (blank MS Word file created on a MacBook, using MS Word 2011 [version 14.7.] followed by a save & close action) (Spennemann and Singh, 2024).

2.3.6. rsid experiment 6

Rsid Experiment 6 examines the effects of blanking out existing text in duplicate file generated by a 'save as' action. As base file served a copy of the final file of rsid experiment 5. This was saved under a new name

Table 3

File creation (C) and edit processes (E) and associated rsid numbers for experiment 5.

Step	rsid	Action
C1	7098170	Sentenced 1 typed out, saved.
C2	9715125	Sentence 1 copied and pasted, then all words 'one' replaced with 'two,' saved
C3	9060201	Text of sentence 1 washed through text editor and 'one' replaced with 'three,' text pasted, saved
C4	9967829	Text of sentence 1 washed through text editor and 'one' replaced with 'four,' text pasted, saved
C5	11814380	Text of sentence 1 washed through text editor and 'one' replaced with 'five,' text pasted, saved
C6	6302464	Text of sentence 1 washed through text editor and 'one' replaced with 'six,' text pasted, saved
C7	4984890	Text of sentences 1–5 washed through text editor and pasted as a block, saved
E1	4341379	Bolded first sentence
E2	15860399	Deleted sentence 3 but left the full stop
E3	2496067	Deleted sentence 6 including full stop
E4	5190171	Deleted last word of sentence 4 with full stop and first word of sentence 5.
E5	13710711	Split sentence 2 after the first comma by adding new line, then deleting text to next comma.
E6	7166929	Italicised the last two words of sentence 5.

Table 4
Synopsis of the text edit experiments reported in this paper.

Exp.	Aim	Method
1	Examines differences between iterative text generation in a single document and inserting iteratively generated text by copying or reading-in data	Three files created on a MacBook (experiment 1A) and Windows PC (experiment 1B). Files 1 and 2 involve pasting iteratively generated text, while file 3 involves reading it in. Uses text from a file with 10 edit stages to create complex rsid structure
2	Compares files with and without Endnote referencing data when inserting prior generated text	Uses base document, generates control text with embedded Endnote references. Two files are created, one with Endnote references and one with references deleted. Conducted on a MacBook (2A, 2B) and a Windows machine (2C, 2D).
3	Assesses the ability to detect differences between typed and pasted text in the same edit session.	Uses base document with zones for typed and pasted text. Pasted text is 'washed' in a text editor to remove xml edit codes
4	Examines the effects of swapping files between computers with different operating systems	Tests two variations - sequential text generation and iterative text generation with track changes. Files copied between a MacBook and a Windows PC
5	Analyzes the effects of complex file creation and editing processes on RSID allocation and persistence	Uses base document and employs various file creation and editing processes (details in Table 3).
6	Investigates the impact of blanking out existing text in a duplicate file created using 'save as.'	Uses a file copy of the final file from 5, blanks out text, and saves it under a new name.
7	Examines whether moving existing text by dragging and dropping or cutting and pasting leaves traces.	Creates four paragraphs in one editing session, then cuts, pastes, and drags and drops one paragraph each
8	Explores whether legacy rsid from source files are added to the rsid listed in the rsidtbl of analysis files	Uses base document pastes the entire text of the current manuscript (170 revisions), and extracts Rsid
9	Assesses the impact of initially generating a document on GoogleDocs on rsid allocation	Examines how GoogleDocs affects rsid allocation during initial document creation and editing
10	Investigates whether using a third-party text editor instead of MS Word changes the structure of MS Word files downloaded from GoogleDocs	Uses MS Word files downloaded from GoogleDocs and analyzes any structural changes

using the 'save as' function. All text was deleted, extra text typed in and the file saved.

2.3.7. rsid experiment 7

Rsid Experiment 7 examines whether the movement of existing text by dragging and dropping or by cutting and pasting would leave tell-tale traces. As base document served a file for developed and reported in a previous paper (blank MS Word file created on a MacBook, using MS Word 2011 [version 14.7.] followed by a save & close action) (Spennemann and Singh, 2024). Four paragraphs were created in the same editing session. The saved file was reopened, one paragraph was cut and pasted at a different location in the document, while a second paragraph was highlighted and dragged and dropped to a different location. This was followed by a save action.

2.3.8. rsid experiment 8

Rsid Experiment 8 examines whether the legacy rsid contained in the *insrsid* and *charrsid* tags of a source file are added to the rsid listed in the *rsidtbl* of the analysis file. As base document served a file for developed and reported in a previous paper (blank MS Word file created on a MacBook, using MS Word 2011 [version 14.7.] followed by a save & close action) (Spennemann and Singh, 2024). A copy was created (analysis file) and the entire text of this current manuscript (as source file), as far as it had been drafted at this point (170 revisions), was pasted

in. For the analysis, all rsid were extracted from the *insrsid* and *charrsid* tags of both the source file and the analysis file. In addition, the rsid contained in the *rsidtbl* of the base document and the analysis file were also extracted (Experiment 8A).

The experiment was repeated using files with less legacy data. As base document served a file for developed and reported in a previous paper (blank MS Word file created on a MacBook, using MS Word 2011 [version 14.7.] followed by a save & close action) (Spennemann and Singh, 2024). The text copied in was sourced from the file created for rsid Experiment 7. As before the *insrsid* and *charrsid* tags of both the source file and the analysis file were extracted (Experiment 8B).

2.3.9. rsid experiment 9

It has become quite common for students to develop assignments using GoogleDocs, making use of the ability to work on the same document in a platform- and location-independent form, for example using a computer at work and one at home. Rsid Experiment 9 assesses to what extent the initial generation of a document on GoogleDocs may have on the allocation of document rsid and rsid allocated during the editing stages. It should be noted that GoogleDocs autosaves the text after a period of short inaction. For the experiment a GoogleDocs file was created that comprised the following editing stages, with each following step only commencing after the automated save action had completed:

- 1) Typing in text of paragraph 1
- 2) Typing in text of paragraph 2
- 3) Typing in text of paragraph 3
- 4) Bolding some words in paragraph 2
- 5) Italicising some words in paragraph 3

The resulting analysis files were generated and saved to the laptop as follows, simulating user actions:

- Content of GoogleDocs file downloaded as a word docx file
- Content of GoogleDocs file downloaded as a rtf file
- Content of GoogleDocs file copied and pasted into a blank MS Word file, using client version on the laptop
- Content of GoogleDocs file copied and pasted into a blank MS Word file, using Online version

The file copies of the GoogleDocs file downloaded as a word docx file were further subjected to the following actions (using a MacBook with MS Word 16.73 [23051401]), simulating user interaction:

- Downloaded file was opened and saved without changes. This simulates the verification that the file looks correct before being passed on or submitted.
- Downloaded file was opened, a word added and saved. This simulates a single, minor change before file was passed on or submitted.

2.3.10. rsid experiment 10

To assess whether using a third-party text editor rather than MS Word would change the structure of the MS Word files downloaded from GoogleDocs. As source file served file copies of the GoogleDocs file downloaded (in MS Word format) as part of rsid Experiment 9. File copies were subjected to the following actions using Apple's TextEdit application on a MacBook:

- Downloaded file was opened and saved without changes
- Downloaded file was opened, a word added and saved, overwriting the existing file
- Downloaded file was opened, a word added and saved under a new file name.

3. Results

The document creation time is frequently advanced as an argument that the document had been created over a prolonged period of time. Likewise, short document creation to document modification intervals are often seen as an indicator of untoward actions. The absolute time and date information as in contained MS Word files and retrievable via the document properties panel has little evidentiary value as it can be easily manipulated by disconnecting a machine from the network and resetting the clock to a desired time and date. Such virtual time travel is documented in the partial screen capture shown in Fig. 3a, where a file was seemingly created on 2 January 1980, four years before the first Apple Macintosh computer was launched on January 24, 1984 (Library, 1983) and five years before the first version of MS Word for the Mac appeared on the market (1985) (Knight, 2013). Fig. 3b shows a file which was created on 1 October 2022, but seemingly had been modified 42 years earlier.

3.1. Revision experiments 1 to 6

Revision Experiment 1 assessed whether pre-existing rsid influenced the number of revisions as displayed in the property window of MS Word files. The pre-existing rsid were not reflected in the number of revisions as displayed. These increased by one following each edit&save action.

Revision Experiment 2 assessed whether save actions without edits influenced the recorded number of revisions as displayed in the property window of MS Word files. Save actions without a prior editing event do not influence the number of revisions, nor the total document editing time, as displayed, and are also not reflected in the time modified entry.

Revision Experiment 3 examined whether times on inaction would influence the total document editing time as displayed in the property window of MS Word files. The total document editing time did not change after the file was saved which had been left open but otherwise left unchanged for 80 min. The duration the file was left open was also not reflected in the time modified entry (experiment 3A). In the file where text was added immediately after opening, but which was then left open for 20 min, the time stamp for the time modified entry changed, but the overall edit time did not.

Revision Experiment 4 examined whether the total document editing time as displayed in the property window of MS Word files reflected actual editing time. The results showed that the actual 1-min time intervals between save actions (following an edit action) are accurately reflected in the total editing time as recorded by MS Word. Once the intervals between save actions increased, the total editing time as

recorded by MS Word diverged increasingly (Table 5). At the end of the experiment, the actual total editing time was 121 % greater than the time recorded by MS Word.

An ancillary observation was the fact that during the 5-min time intervals the revision numbers increased twice without actual save actions being carried out in the file itself. During that time other documents that were being worked on were being saved. Trials with a different document could not replicate these revision number increases. While the follow-up experimentation (revision experiment 5) could not replicate these, it confirmed inconsistent allocation of total editing time by MS word when compared to actual time elapsed.

Revision Experiment 5B tested whether the number of revisions would carry over to a file created via a ‘save as’ action. This was not the case. The revision numbering restarted at ‘1’.

Revision Experiment 6 was designed to assess whether the use of a legacy file would carry over the number of revisions and total document editing time data, or whether the total editing time as displayed would only relate to the file copy and reflect the actual total time elapsed between the copy event and the current last edit. In both cases the action of blanking out existing text with a subsequent save action resulted in an increase of the existing (legacy) number of revisions by one, with adjustment total document editing time by 1 min. The time of the last save was correctly displayed in time modified entry in the property window.

Table 5
Results of revision experiment 4.

Time	Revision number	Total editing time	
		MS Word	actual
16:39	start		
16:40	1	1	1
16:41	2	2	2
16:42	3	3	3
16:43	4	4	4
16:44	5 *	4	5
16:45	7	5	6
16:47	8	7	8
16:49	12**	8	10
16:51	13	10	12
16:56	14	11	17
17:01	15	12	21
17:11	18**	14	31

Notes: *) accidental intermediate save carried; **) no intermediate saves were carried out (see text).

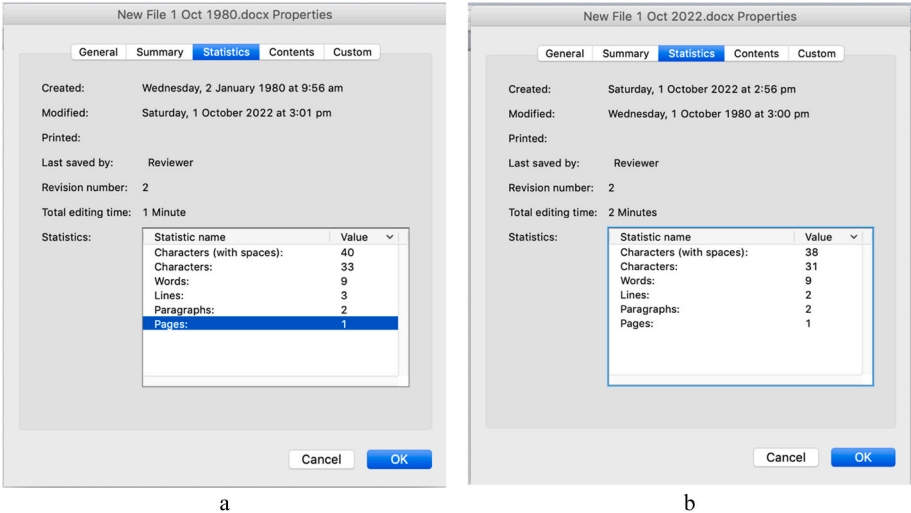


Fig. 3. Examples of digital ‘time travel’ achieved by manipulating the computer clock.

3.2. rsid experiment 1

Experiment 1 considered the insertion of a chunk of iteratively generated control text into blanked files. The control text contained eleven different rsid in a complex rsid structure (Fig. 2). The copy action via the clip board (experiment 1A), reduced the control text to single block of text, enclosed with a new rsid. The outcome was the same irrespective of whether a MacBook or a Widows machine was used. The copy action by reading in ('insert file') the control text (experiment 1B), maintained the complex rsid structure and tags, but replaced each of the eleven different rsid with the same new rsid. Again, the outcome was the same irrespective of whether a MacBook or a Windows machine was used.

3.3. rsid experiment 2

Since all file manipulation processes used file copies of the same base file, the *rootsid* did not undergo a change in any of the files. The file operations on the MacBook resulted in the deletion of the temporary rsid in the sample file and the subsequent creation of a persistent rid as well as another temporary rsid, which is consistent with prior research (Spennemann and Singh, 2024). Neither the copy nor the read-in (insert) process added any of the rsid that had been contained in the generated control text files either in the rsid listing (*rsidtbl*) or the tags.

A comparison of the codes of the text as copied and as read-in to the test files with the code embedded in the control text itself, shows that all legacy edit codes that were contained in by the control text file were carried over into both the copied/read-in files (Fig. 4a–c). Importantly, the *charrsid* of the control text (*charrsid9768227*) was maintained and carried over, both in the pasting and the reading-in process (Fig. 4b and c). The text as read-in added additional codes, not marked with a rsid, which reflects different line breaks. Intriguingly, the line breaks differ between the original (Fig. 4a) and the two copy options (Fig. 4b and c). The insertion of the text, however, was characterised by a new *insrsid* that differed from the that in the legacy file from which the text chunk originated. By comparison, if the control text was not copied directly into the test file but first pasted into a text editor, with that text then pasted into the test file, *all* legacy edit codes were dropped. Also, in that

instance the *insrsid* and the *charrsid* are the same. Running an update of the Endnote references, with no additional references inserted, does not change any of the rsid codes in the text.

3.4. rsid experiment 3

Experiment 3 was designed to assess whether it is possible to detect differences between text that was typed in as opposed to (washed) text that was pasted in within the same edit session (before a save). All inserted text, whether typed or pasted, within a given edit session, is allocated the rsid for that session (Fig. 5). Where text is pasted in, the text is also allocated a *charrsid* with the same number as that edit session (Fig. 5e and f). If text from a preceding edit session was copied in the same fie, it will be allocated a new *insrsid*, but will retain the *charrsid* of that earlier session ()

a	[\rtlch\fcs1 \af31507 \ltrch\fcs0 \insrsid14500334 EDIT SESSION 1 TYPED TEXT \par]
b	[\rtlch\fcs1 \af31507 \ltrch\fcs0 \insrsid11763616 EDIT SESSION 2 TYPED TEXT. \par]
c	[\rtlch\fcs1 \af31507 \ltrch\fcs0 \insrsid1528783 EDIT SESSION 3 TYPED TEXT \par]
d	[\rtlch\fcs1 \af31507 \ltrch\fcs0 \insrsid3161927 EDIT SESSION 4 TYPED TEXT \par]
e	[rtlch\fcs1 \af31507 \ltrch\fcs0 \insrsid11763616\charrsid11763616 EDIT SESSION 2 PASTED EXTERNAL TEXT \par]
f	[\rtlch\fcs1 \af31507 \ltrch\fcs0 \f31575\insrsid1528783\charrsid1528783 EDIT SESSION 3 PASTED EXTERNAL TEXT \par]
g	[\rtlch\fcs1 \af31507 \ltrch\fcs0 \insrsid3161927\charrsid11763616 EDIT SESSION 4 PASTED INTERNAL TEXT \par]

Fig. 5. Allocation of insrsid and charrsid to text written and pasted over four edit&save sessions.

a	[\rtlch\fcs1\af0\ltrch\fcs0\insrsid1592005\charrsid9768227\hich\af39\dbch\af42\loch\f39\hich\f39 A plant collecting craze engulfed Europe in the second part of the nineteenth century, with many bourgeois plant collectors exhibiting their exotic possession in horticultural shows and vying for prizes.\ae\loch\f39 Horticultural establishments in \hich\af39\dbch\af42\loch\f39\hich\f39 Belgium and the South of France were more than happy to service the demand and continually expand their range of offerings\ae\loch\f39. The majority of the novel plants were illustrated in the catalogues of horticultural establishments in form of line drawings in w\hich\af39\dbch\af42\loch\f39 o\hich\af39\dbch\af42\loch\f39\hich\f39 e odcuts and steel plate etchings and subsequently publiscised by republication in horticultural magazines. Photographic representation of plants in nursery catalogues and horticultural magazines did not commence until the mid 1880s\ae\par]
b	[\rtlch\fcs1\af0\ltrch\fcs0\insrsid1455990\charrsid9768227\hich\af39\dbch\af42\loch\f39\hich\f39 A plant collecting craze engulfed Europe in the second part of the nineteenth century, with many bourgeois plant collectors exhibiting their exotic possession in horticultural shows and vying for prizes.\ae\loch\f39 Horticultural establishments in \hich\af39\dbch\af42\loch\f39\hich\f39 Belgium and the South of France were more than happy to service the demand and continually expand their range of offerings\ae\loch\f39 The majority of the novel plants were illustrated in the catalogues of horticultural establishments in form of line drawings in wo\hich\af39\dbch\af42\loch\f39o\hich\af39\dbch\af42\loch\f39\hich\f39 dcuts and steel plate etchings and subsequently publiscised by republication in horticultural magazines Photographic representation of plants in nursery catalogues and horticultural magazines did not commence until the mid 1880s\ae\par]
c	[\rtlch\fcs1\af0\ltrch\fcs0\insrsid14228782\charrsid9768227 \hich\af39\dbch\af42\loch\f39 A\hich\af39\dbch\af42\loch\f39\hich\f39 plant collecting craze engulfed Europe in the second part of the nineteenth century, with many bourgeois plant collectors exhibiting their exotic possession in horticultural shows and vying for prizes.\ae\loch\f39 Horticultural establishments in Belgium and the Sou\hich\af39\dbch\af42\loch\f39 t\hich\af39\dbch\af42\loch\f39\hich\f39 h of France were more than happy to service the demand and continually expand their range of offerings\ae\loch\f39 The majority of the novel plants were illustrated in the catalogues of horticultural establishments in form of line drawings in woodcuts and steel pla\hich\af39\dbch\af42\loch\f39 t\hich\af39\dbch\af42\loch\f39\hich\f39 e etchings and subsequently publiscised by republication in horticultural magazines Photographic representation of plants in nursery catalogues and horticultural magazines did not commence until the mid 1880s\ae\par]
d	[\rtlch\fcs1 \af0 \ltrch\fcs0 \insrsid3693616\charrsid3693616 A plant collecting craze engulfed Europe in the second part of the nineteenth century, with many bourgeois plant collectors exhibiting their exotic possession in horticultural shows and vying for prizes.\ae Horticultural establishments in Belgium and the South of France were more than happy to service the demand and continually expand their range of offerings \ae The majority of the novel plants were illustrated in the catalogues of horticultural establishments in form of line drawings in woodcuts and steel plate etchings and subsequently publiscised by republication in horticultural magazines Photographic representation of plants in nursery catalogues and horticultural magazines did not commence until the mid 1880s \ae\par]

Fig. 4. rsid Experiment 2 MacBook series. Inserted control text. a) control text; b) control text as pasted via the clipboard; c) control text as read-in via 'insert'; d) control text 'washed' by pasting into a text editor and pasting that text into the MS Word file. Note that the rtf code \ae equates to the ® sign in 'normal' text

3.5. rsid experiment 4

Experiment 4 examined the effects of swapping files between computers of different operating systems. The iterative, sequential generation of text (by adding a new line of text at each stage) (experiment 4A) allocated a unique, persistent rsid to each step, but these were allocated with both greater and smaller values (Table 6). The delete and insert actions with track changes turned on (experiment 4B) were also allocated unique, persistent rsid to each step, but, again, these were allocated with both greater and smaller values (Table 7). No systematic differences in rsid allocation could be observed between the Windows (W) and MacOS (M) iterations (Table 6, Table 7).

Once all track changes had been accepted, all tags containing tracking information disappeared, together with all rsid that indicated deletions and all time stamp data. All rsid associated with insertions, however, remained in place.

3.6. rsid experiment 5

Rsid Experiment 5 examined the effects of a complex series of file creation and editing processes on the allocation and persistence of rsid. The comparison of the typed text prior and after deletion events (Fig. 6a vs Fig. 6b) shows that an allocated *insrsid* persists as long as part of the originally inserted text (one or more characters or spaces) are retained. Where the entire text is deleted, the *insrsid* is deleted as well (viz. *insrsid*6302464 in Fig. 6a vs Fig. 6b). Although deleted in the body of the text, the rsid remains retained in the *rsidtbl*.

Modification of inserted text (bolding of first sentence, step E1) is reflected as a modification of the *charrsid* with retention of the prior *insrsid*: *charrsid*9715125 in the original typed text (Fig. 6a) is replaced with *charrsid*4341379 in the edited text (Fig. 6b). The same applies to edit step E6 where text was italicised (original *charrsid*11814380 vs *charrsid*7166929 in edited text, Fig. 6c and d).

3.7. rsid experiment 6

Rsid Experiment 6 examined the effects of blanking out existing text in a duplicate file generated a ‘save as’ action. All pre-existing rsid numbers were carried over. The temporary rsid allocated to the rtf save of the base file was dropped and a new rsid allocated to the inserted text was added. As the document was not closed and reopened between the last save and the save action to rtf, no temporary rsid was allocated.

3.8. rsid experiment 7

Rsid Experiment 7 examined whether the movement of existing text by dragging and dropping or by cutting and pasting would leave tell-tale traces. The internal movement of the text, irrespective of method, is a recorded as a text insert and thus is allocated the *insrsid* of the new editing session, whereas the *charrsid* remains that of the session that created the text. As an entire paragraph was moved, the new paragraph

was also allocated a new *pararsid* tag with the same rsid number as the new editing session.

3.9. rsid experiment 8

Rsid Experiment 8 examined whether the legacy rsid contained in the *insrsid* and *charrsid* tags of a source file are added to the rsid listed in the *rsidtbl* of the analysis file. Given that the analysis file was created as a copy of the base document, it is unsurprising that the *rsidroot* is the same. The sample file dropped one temporary rsid from the Sample File and added one permanent and one temporary rsid, which is in keeping with findings of a previous study (Spennemann and Singh, 2024).

Significantly, *none* of the rsid carried in the copied text were inserted into the rsid table. The analysis of the rsid contained in the *insrsid* and *charrsid* tags showed that the analysis file contained 16 different *insrsid* numbers (associated with 1704 insertions), with the majority (1668) associated with a single *insrsid* number. This contrasts with a total of 2633 different *insrsid* numbers in the source file. While the reduction of *insrsid* numbers in the source file was considerable, that did not carry through the reduction of *charrsid* numbers (133 different numbers in the source file, 120 in the analysis file).

The repeat of the same experiment, using files with fewer legacy data, again showed that *none* of the rsid carried in the copied text were inserted into the rsid table. Importantly, the legacy rsid as contained in *charrsid* tags were retained but were not reflect in the *rsidtbl* of the analysis file.

3.10. rsid experiment 9

Rsid Experiment 9 assessed to what extent the initial generation of a document on GoogleDocs may have on the allocation of document rsid and rsid allocated during the editing stages. The generation of the test document had occurred in five stages with each following step only commencing after GoogleDocs’ automated save action had completed: typing in text of paragraph 1, text of paragraph 2, text of paragraph 3, bolding some words in paragraph 1, italicising some words in paragraph 2.

The first set of sub-experiments assessed how this edit sequence would be reflected in files downloaded from GoogleDocs. Examining the MS Word.docx file which had been created when the GoogleDocs was saved in that format, two significant observations were made: i) the MS Word file lacked a *rsidtbl* in the Settings.XML file; and ii) the five sequential edits remained visible at a fine-grained level with rsid tags, but with ‘00000000’ allocated as the rsid numbers of all edits. Opening a copy of the downloaded Word file and saving under a new name without any other changes generated a root rsid and two additional rsid in the Settings.XML file. While the Document.XML file containing the text showed a reduction of redundant rsid tags (dropping *rsidRPr* and *rsidR* but retaining *rsidRDefault*) and the replacement of most ‘00000000’ tags with the root rsid, the file retained evidence of the original edit sequence. Opening the downloaded file and adding a word followed by a

Table 6
Sequence and relative position of rsid provisioning.

Step	Text	rsid (alphanumeric equivalent)									
1	Mac 1	M1		M1		X					
2	Win 1	M1		M1		X					W1
3	Mac 2	M1		M1		M2	X				W1
4	Win 2	M1	W2	M1		M2	X				W1
5	Mac 3	M1	W2	M1		M2	X		M3		W1
6	Win 3	M1	W2	M1		M2	X		W3	M3	W1
7	Mac 4	M1	W2	M1		M2			W3	M3	X
8	Win 4	M1	W2	M1	X	M2		W4	W3	M3	M4
9	Mac 5	M1	W2	M1		M2		M5	W4	W3	M3
10	Win 5	M1	W2	M1		M2		M5	W5	W4	W3

Abbreviations: M–MacOS; W–Windows OS; X–Temporary rsid (various values).

Table 7

Sequence and relative position of rsid provisioning.

Step	Action	rsid (alphanumeric equivalent)															
1	Sample File Mac	A	A	X													
2	Original Insert	A	A	X												B	
3	Edit Mac 1	A	A	X			W1									B	
4	Edit Win 1	A	A	M1	X		W1									B	
5	Edit Mac 2	A	A	M1	X		W1				W2					B	
6	Edit Win 2	A	A	M1			W1	X		M2		W2				B	
7	Edit Mac 3	A	A	M1			W1		W3	M2	W2	X				B	
8	Edit Win 3	A	A	M1	X		W1		W3	M2	M3	W2				B	
9	Edit Mac 4	A	X	A	M1		W1		W3	M2	M3	W2				B	W4
10	Edit Win 4	A	A	M1	X		W1		W3	M2	M3	W2			M4	B	W4
11	Edit Mac 5	A	A	M1	X	W5	W1		W3	M2	M3	W2			M4	B	W4
12	Edit Win 5	A	A	M1		W5	W1	X	W3	M2	M3	W2	M5		M4	B	W4
13	Changes accepted	C	A	A	M1	W5	W1		W3	M2	M3	W2	M5		M4	B	W4 X

Abbreviations: A- rsid allocated on creation of file; B- rsid allocated to initial text insertion; C- rsid allocated on acceptance of all changes; M- edits with MacOS, W- edits with Windows OS; X- Temporary rsid (various values).

a	[*insrsid7098170\charrsid7098170 Sentence 1] [*insrsid9715125\charrsid9715125 Text of sentence][*insrsid9715125 two][*insrsid9715125\charrsid9715125 , sentence][*insrsid9715125 two][*insrsid9715125\charrsid9715125 , sentence][*insrsid9715125 two][*insrsid9715125\charrsid9715125 , sentence][*insrsid9715125 two][*insrsid9715125\charrsid9715125 , sentence][*insrsid9715125 two][*insrsid9715125\charrsid9715125 and still sentence][*insrsid9715125 two.] [*insrsid9060201\charrsid9060201 Sentence 3] [*insrsid9967829\charrsid9967829 Sentence 4] [*insrsid11814380\charrsid11814380 Sentence 5] [*insrsid6302464\charrsid6302464 Sentence 6]
b	[**insrsid7098170\charrsid4341379 Sentence 1] [*insrsid9715125\charrsid9715125 Text of sentence][*insrsid9715125 two][*insrsid9715125\charrsid9715125 ,][*insrsid13710711 \par][*insrsid9715125\charrsid9715125 sentence][*insrsid9715125 two][*insrsid9715125\charrsid9715125 , sentence][*insrsid9715125 two][*insrsid9715125\charrsid9715125 , sentence two][*insrsid9715125 two][*insrsid9715125\charrsid9715125 and still sentence][*insrsid9715125 two.] [*insrsid9060201 .][*insrsid9967829][*insrsid9967829\charrsid9967829 Text of sentence four, sentence four, sentence four, sentence four, sentence four, and still sentence][*insrsid11814380\charrsid11814380 of sentence five, sentence five, sentence five, sentence five, sentence five, and still][***insrsid11814380\charrsid7166929 sentence five][*insrsid11814380 .]
c	[*insrsid4984890\charrsid4984890 Washed Text Sentences 1-5]
d	[**insrsid4984890\charrsid4341379 Text of washed text one, washed text one, washed text one, washed text one, washed text one, and still washed text one] [*insrsid4984890\charrsid4984890 . Text of washed text two,] [*insrsid13710711 \par] [*insrsid4984890\charrsid4984890 washed text two, washed text two, washed text one two and still washed text two.. Text of washed text four, washed text four, washed text four, washed text four, washed text four, and still washed text of washed text five, washed text five, washed text five, washed text five, washed text five, and still washed] [***insrsid4984890\charrsid7166929 text five][*insrsid4984890\charrsid4984890 .]

Fig. 6. MacBook series. Evidence of successive deletions. a) code of original typed text (Table 3, steps C1–C6); b) code of edited typed text (Table 3, steps E1–E6); c) code of original copied ‘washed’ text (Table 3, step C7); d) code of edited washed text (Table 3, steps E1–E6). Replaced snippets of code: *) \rttlch\fcsl \af31507 \ltrch\fcsl; **) \rttlch\fcsl \ab\af31507 \ltrch\fcsl \b; ***) \rttlch\fcsl \ai\af31507 \ltrch\fcsl \i.

save&close action also resulted in the allocation of a root rsid and two additional rsid to the Settings.XML file. Again, the Document.XML file containing the text showed a reduction of redundant rsid tags and the insertion of the root rsid at several instances of the original edit sequence, as well as the insertion of a new rsid for the added word. When the GoogleDocs file downloaded as a rtf file, a root rsid and one additional rsid were allocated, with the file showing a reduction of redundant rsid tags and the insertion of the root rsid at several instances, retaining evidence of the original edit sequence. The same pattern was observed when that rtf file was opened and saved as a Ms Word.docx file (but was allocated a different root rsid).

The second approach was to copy and paste the text from GoogleDocs file into a blank MS Word file, both using client version and the Online version. In the case of the client version, the Document.XML file retained the fine-grained detail of the edit sequence with all rsid tags (*rsidRDefault*, *rsidRPr* and *rsidR*) reflecting the root rsid of the file. Pasting text from GoogleDocs file into a blank MS Word of the online version also retained the detail of the edit sequence of the GoogleDocs file but the inserted rsid tags uniformly reflected one number for all *rsidRPr* tag and another for all *rsidR* tags. The *rsidRDefault* tag was

dropped.

3.11. rsid experiment 10

The experiment was designed to assess whether using a third-party text editor rather than MS Word would change the structure of the MS Word files downloaded from GoogleDocs. File copies of the GoogleDocs file downloaded in MS Word format were opened in Apple's TextEdit application on a MacBook. The compressed archive of the.docx file generated by TextEdit when the downloaded file was opened and saved, lacked the Settings.XML file. The Document.XML file which originally contained a large number of rsid tags (see experiment 9) was stripped of all rsid tags. The remaining tags provided residual evidence that edit actions (bolding and italicising) had taken place in paragraphs 2 and 3. Opening the file, adding a word and saving it under a new file name, or overwriting a file with the same name, provided a document of similar appearance, devoid of rsid tags but retaining residual evidence that edit actions had taken place.

4. Discussion

When investigating allegations of academic misconduct, in particular contract cheating, it is desirable to be able to independently verify a student's assertion of the document generation and editing processes. The limitations to tracking exact origins of text have been noted in the literature, but primarily from the angle of document comparison and integration (Filho, 2017; Filho et al., 2017), rather than digital forensics. While the extant literature noted the potential usefulness of rsid numbers for the detection of academic plagiarism, in particular contract cheating in general (Johnson and Davies, 2020; Johnson et al., 2020, 2022; Johnson et al., 2022), these papers erroneously assumed that MS Word would adhere to the ECMA specifications for the Office Open XML standard (Spennemann and Singh, 2024). Castiglione et al., claim that MS Word tracks all insertions and deletions, including the author attributes, allocating rsid numbers in the process (Castiglione et al., 2017). While this holds true for documents where the track changes features have been turned on, it does not apply to documents that are edited where the review function has not been activated.

Given that most student submissions are not submitted with the review function activated and, given that MS Word does not adhere to the ECMA specifications, the authors carried out a range of experiments to ascertain whether and how common edit and document generation actions manifest themselves in the encoding that underlies all MS Word documents. The underlying premise for much of document analysis is that document authors will develop their manuscript in an iterative fashion, comprised of a sequence of writing and editing (word & text deletion or insertion through movement), rather than writing a final, word-perfect text. The second premise is that the manuscript will have been created with repeated edit&save actions, especially if it was generated over the course of several days. The document properties (file 'core.xml' in the 'docProps' folder of the XML package) allows to assess the time elapsed between document generation and the last edit (also accessible with the properties window of the file in the MS Word).

For an examination of the evidence for the generation and editing of MS Word documents to be meaningful for the investigation of alleged academic misconduct, it is important that as much detail as possible has been gleaned by way of interview about the alleged perpetrator's document generation and edit processes (see Appendix B), as this allows a forensic examination to turn these stated processes into scenarios that can be tested for veracity. The following investigative scenarios are based on real world examples that have been derived from personal experience of two of the authors (DHRS & CLS) as well as the experiences of colleagues, all of whom have had or currently have involvement with academic/student misconduct investigations at our institution.

Scenario 1: the properties panel of the document submitted for analysis shows that the document was recently created, has only a short editing time and a low number of revisions

Given the revision experiments reported earlier, a short editing time and a low number of revisions can be indicative of untoward action. Such property data can be generated when a file that contained numerous edits is saved under a new name or if text from another file is copied in toto into a blank file and saved. In the first instance, all pre-existing (legacy) rsid are carried over from the old to the new document, which includes the *rootrsid*, all rsid listed in *rsidtbl* and all rsid associated with edit actions in the text itself. If the examination of the files that these legacy rsid are lacking, if there are discrepancies between the rsid listed in *rsidtbl* and the rsid contained in document tags, or if there is a large body of text without variation in *insrsid* and *charrsid*, then the evidence indicative of untoward action will follow patterns as described in scenario 3.

Short editing time alone can only be seen as indicative, however, as the time stamping of the total editing time, as reported as difference between the document creation and the document modification date can

be inaccurate. While it had been noted elsewhere that MS Word only records time stamps in full minutes, omitting the seconds (Spennemann and Spennemann, 2023), compacted and non-linear lag in elapsed editing time has been observed in experiment 4. Of importance here is that time seems to be more compressed in instances of longer time gaps between save actions (Table 5).

Scenario 2: the properties panel of the document submitted for analysis shows that the document was been created some time ago, exhibits a considerable editing time and shows a high number of revisions

While *prima facie* the prolonged editing time and high number of revisions seems to suggest genuine document generation, it must be noted that, given the revision experiments reported earlier, it is possible to blank out a copy of an older document and thus carry over the document generation information of that file. If genuine, it can be expected that the total number of rsid associated with text insertion and edit events is largely equivalent to the total number of rsid listed in the *rsidtbl*, especially if the document was created on a windows machine. Moreover, all, or at least the overwhelming majority of *insrsid* and *charrsid* numbers in the text body should be reflected in the *rsidtbl*. See scenario 3.

If the document was reputedly generated on a Windows machine and the number of rsid considerably exceeds the total number of rsid associated with text insertion and edit events, then it is likely that an existing document was used with its original text blanked, thereby retaining the original document creation date and number of revisions made until that point and expands on the latter as new edits occur. This can be proven if as freshly created file (generated on the same machine) can also be obtained for comparison, in order to identify which legacy rsid are embedded in the .dot/.dotm template file.

Scenario 3: the document submitted for analysis was stated to have been developed over time, and no material is claimed to have been copied from other files

An iterative process of adding and deleting text will assign a different *insrsid* for each event that ends with a save action. These will be retained in the *rsidtbl* of the document. A genuine iteratively generated manuscript will exhibit numerous *insrsid* codes that represent that author's writing and editing progress similar to the example shown in Fig. 2. If an inserted section of text contains additional snippets of edit and insertion codes that are associated with a *charrsid* not listed in the *rsidtbl* (e.g. Fig. 4b and c), then that chunk of text was copied from another MS Word file which contained that iteratively generated text.

If an inserted section of text lacks additional snippets of edit or insertion codes altogether, then that total section of text was either written in its entirety during that edit session or it was pasted in from another source. Such rsid-less, 'washed' text could have come from i) text lifted from a webpage or pdf document, ii) text copied from an e-mail, iii) text copied from the output of a paraphrasing tool, or iv) text copied from the output of a generative Artificial intelligence program. It also possible that the rsid-less text chunk was generated in a text editor before being pasted in.

Pasting in from another source can be distinguished from extended typing by the presence of a *charrsid* that follows the *insrsid* (Fig. 5). Directly typed text, on the other hand, is only preceded by the *insrsid* of that editing session. The *charrsid* will be the same as the *insrsid* where the font (type, character size) of the pasted legacy text has been adjusted to conform with the format into which the text has been pasted (during the same edit& save session. The *charrsid* will differ if the font (type, character size) of the pasted legacy text has not been modified.

The copying and insertion of new text into files leaves tell-tale traces of the editing process. A chunk of text that had been generated over a series of edit&save sessions, and that had undergone a series of textual refinements, will contain a complex and often nested set of tags, with

associated rsid numbers, that endure the copying process unless the text has been ‘washed’ through a text editor (Fig. 4d). Importantly, text copied from a different word file will be enclosed with two rsid tags, the *insrsid* tag that denotes the paste action itself and is associated with the then current edit session, and the *charrsid* tag that denotes the past edit actions of the text that had been pasted in. Both rsid numbers must be present in the *rsidtbl* (rsid Exp 8) if the paste action involved text that already existed in the current document and where the author merely rearranged the order of text. If the inserted text lacks a *charrsid* tag, then the text was moved during that edit session.

A mismatch between the *insrsid* and *charrsid* tags of the same paragraph should trigger a cross check with the rsid listed in the *rsidtbl*. If the rsid number of one or more *charrsid* tags is missing from the *rsidtbl*, then the text enclosed in those *charrsid* tags was copied or imported from a different word document. It should be noted that mere mismatch between *insrsid* and *charrsid* tags (which are both listed in the *rsidtbl*) does not indicate pasting of foreign text, as such a mismatch can also occur when some of the character formatting is changed in a different edit&save session.

If an entire word document, for example a contract-written essay, were to be copied into the clipboard and pasted into new Word document, then all *insrsid* tags of the inserted text would be same, while the *charrsid* tags might vary. If an output of a generative Artificial intelligence program were to be pasted *en bloc* into a new Word document, then the entire text would be devoid of rsid and be enclosed with a single set of *insrsid* and *charrsid* tags. In both instances, any additional framing, such as author attribution or (assignment) title, would appear different as they will have the same *insrsid* and *charrsid* tags of that, or a subsequent, editing session.

Finally, some comments with regard to the insertion of rsid-less text are warranted at this point. While the origin of the inserted section of text cannot be ascertained from the rsid codes alone, the nature and extent of the insertion can be determined. Despite this caveat, forensic examination of the text should consider whether the insertion of rsid-less text is limited to a single paragraph in a single edit&save session, whether multiple sections of rsid-less text have been inserted in the single edit&save session at different parts of the document; and whether multiple sections of rsid-less text have been inserted in multiple edit&save sessions. While innocuous explanations are possible, insertions of single paragraph segments of rsid-less text, that do not raise any similarity flags in programs such as Turnitin are most likely the product of a paraphrasing tool. This can be verified by comparing text in question with the original text as indicated by the reference (if given). While paraphrasing tools change the sentence structure, they will not alter the flow and sequence of the argument.

It is now possible from the analysis of the rsid whether a section of text has been written by a generative Artificial intelligence program. The insertion of multiple rsid-less sections of text in the same edit&save session, however, may well point to the use of the output of a generative Artificial intelligence program and its insertion at various points of the document. While various tools exist to detect Artificial intelligence generated text, they have varied levels of accuracy (Khalil and Er, 2023; Ventayen, 2023; Uzun, 2023; Habibzadeh, 2023) and usually rely on extended sections of text or whole documents. If the use of such AI detection tools is to be contemplated, it is recommended that the rsid-less sections of text are collated into a new ‘clean’ document and that this document is fed into the detection tool.

Scenario 4: the document contains chunks of text that appear plagiarised but show substituted words or part sentences and/or uses British instead of US American spelling

The copied section of text will be enclosed by an *insrsid* tag that presents the initial copying event. If the text was taken from another word document, then the *insrsid* and *charrsid* tags of the section will differ (with the *charrsid* reflecting the legacy rsid). If the document

author carried out subsequent changes, such as word substitution or spellcheck (for example to correct US American with British spelling) in the same edit&save session, then this the substitutions and spelling correction will be enclosed with a pair of *insrsid* and *charrsid* tags carrying the same session rsid as the paste event. If the substitution or spellcheck was carried out at a later edit&save session, then the *charrsid* tags will be the same as that of the initial paste action, but the *insrsid* will reflect that of the later edit&save session.

Scenario 5: the document contains a chunk of text that appears plagiarised with a spurious reference at the end and a spurious reference breaking up the text

The copied text will be enclosed by an *insrsid* tag that presents the initial copying. If the references were inserted in the same edit&save session, then these references will be enclosed with a pair of *insrsid* and *charrsid* tags carrying the same session rsid as the paste event. If the references were inserted during a later edit&save session, then the *charrsid* tags will be the same as that of the initial paste action, but the *insrsid* will reflect that of the later edit&save session. If the insertion of the reference breaking of the plagiarised text occurred at later edit session than the insertion of the first reference, then the *charrsid* tags will be the same as that of the initial paste action, but the *insrsid* will reflect that of the later edit&save session and thus differ from the rsid of the first reference insertion.

Scenario 6: the document shows observable inconsistencies in quality or style suggesting multiple authorship

The investigation of this requires a comparison of the editing characteristics of the paragraphs that differ in quality or style. If they were written by the same author, the text creation and editing pattern should be the same or very similar. The pasting of pre-existing text from a different document can be detected looking for differences between the *insrsid* and the *charrsid* that follows (see scenario 5).

Scenario 7: the document supplied is an old, early draft, which is provided as evidence of prior own work

For this analysis two versions will be in hand. Setting aside issues of creation and modification dates that can be manipulated (see above), all rsid listed in the *rsidtbl* of the draft version must also be present in the submitted version. As the final version would comprise at least one more edit&save action, the number of rsid in the final document must be greater than that of the draft document. These additional rsid must be associated with edits and additional text that is not included in the draft. If, instead the ‘draft’ version was created by reverse-editing the final copy, the number of rsid in the *rsidtbl* of the draft version is likely to be as high or higher than the submitted copy and will differ by at least one rsid from the final version.

5. Conclusions

This study has shown that revision save identifiers (rsid) are a powerful tool for document analysis as they will leave traces in MS Word files which provide evidence of the creation and editing processes used to shape a specific document. These traces include the ability to detect differences between iterative text generation in a single document and inserting iteratively generated text by copying or reading-in data; between typed and pasted text in the same edit session, and between existing text that was moved dragging and dropping as opposed to cutting and pasting. In addition, traces of legacy edit events and legacy rsid (in *charrsid* tags but not *rsidtbl*) are retained when inserting prior generated text. In itself, however, such evidence only illuminates the document generation process, but does not provide evidence of academic misconduct.

To be meaningful, it is necessary to glean, by way of interview with the alleged perpetrator, as much detail as possible about the asserted generation and edit processes that led to the document to be examined. [Appendix B](#) provides a suggested set of interview questions as well as the rationale why this information is useful. A forensic examination of the rsid patterns in the disputed document allows to test the veracity of the author's assertions, thereby confirming or refuting allegations of academic misconduct. It should be noted, however, that this validation of allegations of misconduct is merely tool and as such entirely separate from any educative or punitive action that may flow from these findings.

CRediT authorship contribution statement

Dirk HR. Spennemann: Writing – review & editing, Writing – original draft, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Rudolf J. Spennemann:** Writing – review & editing, Investigation, Formal analysis. **Clare L. Singh:** Writing – review & editing, Investigation.

Appendix A. How to access rsid numbers

Method A: rsid in hexadecimal format

Every file in Word.docx format is in fact a compressed archive of multiple files, which can be easily accessed by replacing the '.DOCX' extension of the file with '.ZIP' and then extracting that archive using standard decompression/de-archiving software. The resulting file folder contains three subfolders. The required files are located in the 'word' subfolder. All rsid numbers are rendered as hexadecimal characters (e.g. rsid004C4A41) and stored in the file *settings.xml*. The body of the text document, is stored in the file *document.xml* where all editing and formatting tags carry rsid numbers (as hexadecimal characters) that correspond with some of the rsid listed in the *settings.xml* file. As MS Word cannot open these files, they need to be read using a plain text editor such as TextEdit or Notepad. All rsid numbers contained in a given document are stored in a summary table ('rsidtbl') which lists them in increasing numerical order. In addition, the tag 'rsidroot' shows the rsid allocated to the document itself.

Method B: rsid in whole number format

The benefit of this approach is that all rsid numbers are rendered as standard numerical strings (e.g. rsid4999745) which are more comprehensible for the lay person than numbers in hexadecimal format (i.e. rsid004C4A41). The process requires the user to save the file in rich text format (.rtf), to open that file in MS Excel or a Text Editor that does not attempt to interpret the rtf file, delete the first line of code and then to save the file as text (.txt). Thereafter the coding of the file can be read using MS Word or a plain text editor such as TextEdit or Notepad. All rsid numbers contained in a given document are stored in a summary table ('rsidtbl') which lists them in increasing numerical order. In addition, the tag 'rsidroot' shows the rsid allocated to the document itself. It should be noted that exporting to rtf adds a temporary rsid (reflecting this save action, which not contained in the xml file. This does not affect the interpretation of any of the rsid in the text edit sequences.

Appendix B. Suggested questions to elicit information of about an author's document generation processes for later testing and verification

Question	Rationale
I am going to ask you a number of questions to understand how you write and edit word documents. We are asking the same questions of everybody, so some may not seem to apply to you	
On which computer do you write your assignments?	Windows machines only generate 'fresh' rsid when a new file is created, while Macintosh machines will include all legacy rsid contained in the template file.
Do you have more than one computer?	
If yes: do you use the other machine(s) also for assignment writing?	
Are you currently on your computer?Is that the same computer on which you created typed the assignment?	This can be stipulated when arranging the interview
If yes: Can you please open an new word document right now and type the following text?	To obtain a clean document that contains all legacy rsid that maybe contained in the.dot/.dotm template file
[Integrity Officer to stipulate a short sentence to verify that the document was written at this point in time]	To ensure that the document is created in real time and not created on another machine or by someone else
Can you please e-mail this document to me while we speak?	To obtain this document in real time
Describe to me how you normally work on an assignment. Do you draft the assignment by hand, or do you start straight on the computer?	To ascertain edit sequences that should be visible in the file. Assignments that are created only on a computer will show more complex edit patterns than assignments that are first drafted by hand.
When you write an assignment, do you start with a clean file, or do you reuse an existing file?	To ascertain is legacy rsid may be present in files
When you write an assignment, how often do you save the file?	To establish frequency of save actions, which should be reflected in the rsid pattern

(continued on next page)

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

The authors are indebted to a number of colleagues who shared their experiences which allowed the authors to discuss a range of scenarios: Lachlan Brown (Academic Integrity Officer, Faculty of Arts and Education, CSU); Daniel Cohen (Academic Integrity Officer, Faculty of Arts and Education, CSU); Matthew Ireland (Academic Integrity Officer, Faculty of Science and Health, CSU).

(continued)

Question	Rationale
Do you write all of your assignment in the same file, or do you have several files, one for each part of the assignment?	To understand whether assignments could be merged from different files (which will be reflected in the rsid pattern)
When it is all done, do you submit that manuscript version, or do you create a new one file just for submission?	To understand to whether this is the original of a secondary file (relevance in next question)
[If you create a new file just for submission, how do you do that?]	Files duplicated via save as will have a rsid different signature than files that are generated by copying the text in to an existing file (such as an assignment template)
When you work on assignments, do you back up your drafts (and if so, how)?	To obtain copies of drafts that the comparison of root rsid and edit rsid to track the document's creation over time
Do you have draft copies, or older versions, that you can share with me?	
How many days do you think it took you to write the assignment on the computer?	To correlate with the time stamps

References

- Ahsan, K., Akbar, S., Kam, B., 2022. Contract cheating in higher education: a systematic literature review and future research agenda. *Assess Eval. High Educ.* 47 (4), 523–539.
- Awasthi, S., 2019. Plagiarism and academic misconduct: a systematic review. *DESIDOC. J. Libr. Inf. Technol.* 39 (2), 94–100.
- Bretag, T., Harper, R., Burton, M., Ellis, C., Newton, P., Rozenberg, P., Saddiqui, S., van Haeringen, K., 2019a. Contract cheating: a survey of Australian university students. *Stud. High Educ.* 44 (11), 1837–1856.
- Bretag, T., Harper, R., Burton, M., Ellis, C., Newton, P., van Haeringen, K., Saddiqui, S., Rozenberg, P., 2019b. Contract cheating and assessment design: exploring the relationship. *Assess Eval. High Educ.* 44 (5), 676–691.
- Campbell, C., Waddington, L., 2020. Academic integrity strategies: positive, preventative and punitive. 6th International Conference PAEB 2020 First Virtual ENAI Conference Proceedings, p. 53.
- Castiglione, A., D'Alessio, B., De Santis, A., Palmieri, F., 2017. New steganographic techniques for the OOXML file format. International Conference on Availability, Reliability, and Security. Springer, pp. 344–358.
- Chaudhry, I.S., Sarwary, S.A.M., El Refae, G.A., Chabchoub, H., 2023. Time to revisit student's performance evaluation approach in higher education sector in a new era of ChatGPT—a case study. *Cogent. Educ.* 10 (1), 2210461.
- Choi, J., Park, J., Lee, S., 2021. Forensic exploration on windows file history. *Forensic Sci. Int.: Digit. Invest.* 36, 301134.
- Clare, J., Walker, S., Hobson, J., 2017. Can we detect contract cheating using existing assessment data? Applying crime prevention theory to an academic integrity issue. *Int. J. Educ. Integr.* 13 (1), 1–15.
- Cotton, D.R., Cotton, P.A., Shipway, J.R., 2023. Chatting and cheating: ensuring academic integrity in the era of ChatGPT. *Innovat. Educ. Teach. Int.* 1–12.
- Didriksen, E., 2014. Forensic Analysis of OOXML Documents, Department of Computer Science and Media Technology. Gjøvik University College, Gjøvik.
- ECMA, 2016. Office open XML file formats — fundamentals and markup language reference [ECMA-376-1:2016]. <http://www.ecma-international.org/publications/standards/Ecma-376.htm>. (Accessed 15 November 2021).
- Filho, A.A., 2017. Improving Version-Aware Word Documents. University of Wisconsin, Milwaukee, Milwaukee.
- Filho, A.A., Munson, E.V., Thao, C., 2017. Improving version-aware word documents. Proceedings of the 2017 ACM Symposium on Document Engineering, pp. 129–132.
- Fu, Z., Sun, X., 2015. 基于冗余属性的OOX文本数字水印算法 [Digital watermarking algorithm based on redundant attributes for OOX documents]. in Chinese Nanjing Xinxing Gongcheng Daxue Xuebao 7 (1), 40–45.
- Fu, Z., Sun, X., Liu, Y., Li, B., 2011. Forensic investigation of OOXML format documents. *Digit. Invest.* 8 (1), 48–55.
- Fu, Z., Sun, X., Xi, J., 2015. Digital forensics of Microsoft Office 2007–2013 documents to prevent covert communication. *J. Commun. Network.* 17 (5), 525–533.
- Garfinkel, S.L., Migletz, J.J., 2009. New xml-based files implications for forensics. *IEEE Secur. Priv.* 7 (2), 38–44.
- Habibzadeh, F., 2023. GPTZero performance in identifying artificial intelligence-generated medical texts: a preliminary study. *J. Kor. Med. Sci.* 38 (38).
- Harper, R., Bretag, T., Rundle, K., 2021. Detecting contract cheating: examining the role of assessment type. *High Educ. Res. Dev.* 40 (2), 263–278.
- Hill, G., Mason, J., Dunn, A., 2021. Contract cheating: an increasing challenge for global academic community arising from COVID-19. *Res. Pract. Technol. Enhanc. Learn. (RPTEL)* 16 (1), 1–20.
- ISO/IEC, 2008. Information Technology — Document Description and Processing Languages — Office Open XML File Formats — Part 1: Fundamentals and Markup Language Reference. International Organization for Standardization/International Electrotechnical Commission, p. 5560.
- ISO/IEC, 2016. Information Technology — Document Description and Processing Languages — Office Open XML File Formats — Part 1: Fundamentals and Markup Language Reference. International Organization for Standardization/International Electrotechnical Commission, p. 5024.
- Jeong, D., Lee, S., 2017. Study on the tracking revision history of MS Word files for forensic investigation. *Digit. Invest.* 23, 3–10.
- Johnson, C.S., Davies, R., 2020. Using digital forensic techniques to identify contract cheating: a case study. *J. Acad. Ethics* 18 (2), 105–113.
- Johnson, C.S., Davies, R., 2020. Plagiarism from a digital forensics perspective. In: Hill, C., Khan, Z.R., Kralikova, V. (Eds.), 6th International Conference PAEB 2020 First Virtual ENAI Conference Proceedings, pp. 37–41.
- Johnson, C., Davies, R., Reddy, M., 2022. Using digital forensics in higher education to detect academic misconduct. *Int. J. Educ. Integr.* 18 (1), 12.
- Joun, J., Han, J., Jung, D., Lee, S., 2018. Study on history tracking technique of the document file through RSID analysis in MS word. *J. Korea. Inst. Inf. Secur. Cryptol.* 28 (6), 1439–1448 [in Korean].
- Joun, J., Chung, H., Park, J., Lee, S., 2021. Relevance analysis using revision identifier in MS word. *J. Forensic Sci.* 66 (1), 323–335.
- Khalil, M., Er, E., 2023. Will ChatGPT G et You Caught? Rethinking of Plagiarism Detection. International Conference on Human-Computer Interaction. Springer, pp. 475–487.
- Knight, D., 2013. Software FAQs. Microsoft word for mac FAQ. <https://lowendmac.com/2013/microsoft-word-for-mac-faq/>. (Accessed 5 October 2022).
- Langweg, H., 2012. OOXML file analysis of the July 22nd terrorist manual. IFIP International Conference on Communications and Multimedia Security. Springer, pp. 195–197.
- Apple computer inc, "Macintosh product introduction plan," typed ms. In: Library, S.U. (Ed.), 1983. Apple Computer Inc. Papers, Department of Special Collections. Stanford University Library, 1983.
- Mahlow, C., Piotrowski, M., 2022. Academic writing and publishing beyond documents. Proceedings of the 22nd ACM Symposium on Document Engineering, pp. 1–4.
- Miles, P.J., Campbell, M., Ruxton, G.D., 2022. Why students cheat and how understanding this can help reduce the frequency of academic misconduct in higher education: a literature review. *J. Undergrad. Neurosci. Educ.* 20 (2), A150–A159.
- Miron, J., McKenzie, A., Eaton, S.E., Stoesz, B., Thacker, E., Devereaux, L., Persaud, N., Steeves, M., Rowbotham, K., 2021. Academic integrity policy analysis of publicly-funded universities in Ontario, Canada: a focus on contract cheating. *Can. J. Educ. Adm. Pol./Revue. Can. Adm. politiq. éducation* 197, 62–75.
- Naidu, K., Sevnanarayan, K., 2023. ChatGPT: an ever-increasing encroachment of artificial intelligence in online assessment in distance education. *Online J. Commun. Media Technol.* 13 (1), e2023xx.
- Newton, P.M., 2018. How common is commercial contract cheating in higher education and is it increasing? A systematic review. *Frontiers in Education. Frontiers Media SA*, p. 67.
- Park, C., 2003. In other (people's) words: plagiarism by university students—literature and lessons. *Assess Eval. High Educ.* 28 (5), 471–488.
- Perkins, M., Roe, J., Postma, D., McGaughan, J., Hickerson, D., 2023. Game of Tones: Faculty Detection of GPT-4 Generated Content in University Assessments arXiv preprint arXiv:2305.18081.
- Ries, T., 2017. Filologia i proces pisania cyfrowego—metody i wyzwania. *Wielogłos* (31), 103–127.
- Spennemann, D.H.R., 2023a. Exploring Ethical Boundaries: Can ChatGPT Be Prompted to Give Advice on How to Cheat in University Assignments?, pp. 1–14. Preprint.
- Spennemann, D.H.R., 2023b. ChatGPT and the generation of digitally born “knowledge”: how does a generative AI language model interpret cultural heritage values? *Knowledge* 3 (3), 480–512.
- Spennemann, D.H.R., Biles, J., Brown, L., Ireland, M.F., Longmore, L., Singh, C.J., Wallis, A., Ward, C., 2024. ChatGPT giving advice on how to cheat in university assignments: how workable are its suggestions? *Interactive Technology and Smart Education*. <https://doi.org/10.1108/ITSE-10-2023-0195>.
- Spennemann, D.H.R., Singh, C., 2024. The generation of revision identifier (rsid) numbers in MS Word—implications for document analysis. *Int. J. Digit. Curation* 18 (1), 1–22.
- Spennemann, D.H.R., Spennemann, R.J., 2023. Establishing genealogies of born digital content: the suitability of revision identifier (rsid) numbers in MS Word for forensic enquiry. *Publications* 11 (35), 1–25.
- Uzun, L., 2023. ChatGPT and academic integrity concerns: detecting artificial intelligence generated content. *Lang. Educ. Technol.* 3 (1).
- Ventayen, R.J.M., 2023. OpenAI ChatGPT-generated results: similarity index of artificial intelligence-based contents. International Conference on Soft Computing for Security Applications. Springer, pp. 215–226.
- Weber-Wulff, D., Anohina-Naumeca, A., Bjelobaba, S., Foltýnek, T., Guerrero-Dib, J., Popoola, O., Šigut, P., Waddington, L., 2023. Testing of Detection Tools for AI-Generated Text arXiv preprint arXiv:2306.15666.